

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

КУРСОВОЙ ПРОЕКТ
по дисциплине
«Сетевое программное обеспечение»

по теме:
МИКРОСЕРВИСНАЯ СИСТЕМА УПРАВЛЕНИЯ КОММЕНТАРИЯМИ

Студент:
Группа ИА331

Р. К. Рубцов

Преподаватель:
Лектор и практик

А. В. Лошкарев

Новосибирск 2026 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
2	АРХИТЕКТУРА	4
2.1	Компоненты	4
2.2	Схема взаимодействия	4
2.3	Технологический стек	5
3	РЕАЛИЗАЦИЯ СЕРВИСОВ	6
3.1	Comments Service	6
3.2	Users Service	6
3.3	gRPC-контракт	7
4	API GATEWAY (NGINX)	8
5	DOCKER COMPOSE	9
6	KUBERNETES	10
6.1	Deployment	10
6.2	Service	11
7	СБОРКА И ЗАПУСК	12
7.1	Локальный запуск	12
7.2	Проверка	12
8	ЗАКЛЮЧЕНИЕ	13

1 ВВЕДЕНИЕ

Данный отчёт описывает выполнение финального проекта по курсу «Сетевое программное обеспечение». Проект представляет собой микросервисную систему управления комментариями (код проекта: `comments-s19`), состоящую из двух сервисов и API Gateway на Nginx.

2 АРХИТЕКТУРА

2.1 Компоненты

Финальный проект включает следующие компоненты:

- **Comments Service** (порт 8138 REST) — основной сервис, реализующий CRUD операции для комментариев на FastAPI.
- **Users Service** (порты 8090 REST, 50051 gRPC) — вспомогательный сервис для управления пользователями на FastAPI, также предоставляет gRPC endpoint для проверки существования пользователя.
- **API Gateway** (Nginx, порт 8080) — единая точка входа, маршрутизация запросов к микросервисам.

2.2 Схема взаимодействия

Client -> Nginx (:8080)

 /api/comments/* -> Comments Service (:8138)

 /api/users/* -> Users Service (:8090)

Comments Service -> Users Service (gRPC :50051)

 CheckUserExists(username)

2.3 Технологический стек

Компонент	Технология	Обоснование
Язык	Python 3.11	Скорость разработки, асинхронность
Веб-фреймворк	FastAPI	Высокая производительность, Pydantic валидация
API Gateway	Nginx	Лёгкий reverse-proxy
Протокол внешний	REST (JSON)	Универсальность
Протокол межсервисный	gRPC	Высокая производительность, строгая типизация
Контейнеризация	Docker (multistage)	Малый размер образов
Оркестрация	Docker Compose	Локальный запуск

3 РЕАЛИЗАЦИЯ СЕРВИСОВ

3.1 Comments Service

Сервис реализует полный CRUD для комментариев. Каждый комментарий содержит поля `id`, `author`, `text`. Данные хранятся in-memory. Эндпоинты:

- GET `/api/comments` — список всех комментариев
- POST `/api/comments` — создание комментария (201 Created)
- GET `/api/comments/{id}` — получение по ID
- PUT `/api/comments/{id}` — обновление комментария
- DELETE `/api/comments/{id}` — удаление (204 No Content)

При создании комментария выполняется gRPC-запрос к Users Service на проверку существования автора (`CheckUserExists`). Если пользователь не найден, возвращается 400 Bad Request. При недоступности Users Service — 503 Service Unavailable. При отсутствии ресурса возвращается 404. При невалидных данных — 422.

3.2 Users Service

Сервис управляет пользователями с полями `id`, `name`, `email`. Поддерживает:

- GET `/api/users` — список пользователей
- POST `/api/users` — создание пользователя
- GET `/api/users/{id}` — получение по ID

Сервис также запускает gRPC-сервер на порту 50051, реализующий метод `CheckUserExists`, который проверяет, зарегистрирован ли пользователь с указанным именем. REST и gRPC серверы работают параллельно в одном процессе.

3.3 gRPC-контракт

Межсервисное взаимодействие описано в `proto/user_check.proto`:

```
1 syntax = "proto3";
2 package usercheck;
3
4 service UserChecker {
5     rpc CheckUserExists (UserExistsRequest)
6         returns (UserExistsResponse);
7 }
8
9 message UserExistsRequest {
10     string username = 1;
11 }
12
13 message UserExistsResponse {
14     bool exists = 1;
15 }
```

На основе этого файла с помощью `grpc_tools.protoc` генерируются Python-модули, которые импортируются в обоих сервисах:

- **Users Service** — реализует `UserCheckerServicer` и запускает gRPC-сервер в фоновом потоке.
- **Comments Service** — создаёт `UserCheckerStub` (клиент) и вызывает `CheckUserExists` перед созданием комментария.

4 API GATEWAY (NGINX)

Настроен reverse-proxy. Конфигурация маршрутизации:

```
1 upstream comments-upstream {
2     server comments-svc-s19:8138;
3 }
4
5 upstream users-upstream {
6     server users-svc-s19:8090;
7 }
8
9 server {
10     listen 80;
11     server_name localhost;
12
13     location /api/comments {
14         rewrite ^/api/comments(.*)$ /comments$1 break;
15         proxy_pass http://comments-upstream;
16     }
17
18     location /api/users {
19         rewrite ^/api/users(.*)$ /users$1 break;
20         proxy_pass http://users-upstream;
21     }
22 }
```


5 DOCKER COMPOSE

Проект запускается одной командой. Файл `docker-compose.yml` описывает три сервиса:

```
1 services:
2   comments-svc-s19:
3     build: ./services/comments-service
4     container_name: comments-svc-s19
5     expose: ["8138"]
6     healthcheck:
7       test: ["CMD", "curl", "-f", "http://localhost:8138/health"]
8     networks: [app-net]
9
10  users-svc-s19:
11    build: ./services/users-service
12    container_name: users-svc-s19
13    expose: ["8090", "50051"]
14    healthcheck:
15      test: ["CMD", "curl", "-f", "http://localhost:8090/health"]
16    networks: [app-net]
17
18  gateway:
19    image: nginx:alpine
20    container_name: gateway-s19
21    ports: ["8080:80"]
22    volumes:
23      - ./infra/nginx.conf:/etc/nginx/conf.d/default.conf
24    networks: [app-net]
25
26 networks:
27   app-net:
28     driver: bridge
```

6 KUBERNETES

Для проекта также подготовлены манифесты Kubernetes, описывающие развёртывание Comments Service:

6.1 Deployment

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: comments-app
5 spec:
6   replicas: 2
7   selector:
8     matchLabels:
9       app: comments-app
10  template:
11    metadata:
12      labels:
13        app: comments-app
14    spec:
15      containers:
16      - name: comments-container
17        image: comments-s19:latest
18        ports:
19        - containerPort: 8138
20      resources:
21        requests:
22          memory: "64Mi"
23          cpu: "250m"
24        limits:
25          memory: "128Mi"
26          cpu: "500m"
27      livenessProbe:
28        httpGet:
29          path: /health
30          port: 8138
31        initialDelaySeconds: 5
32        periodSeconds: 10
33      readinessProbe:
34        httpGet:
```

```
35         path: /health
36         port: 8138
37     initialDelaySeconds: 2
38     periodSeconds: 5
```

6.2 Service

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: comments-svc-s19
5 spec:
6   selector:
7     app: comments-app
8   ports:
9     - protocol: TCP
10       port: 80
11       targetPort: 8138
12   type: ClusterIP
```

7 СБОРКА И ЗАПУСК

7.1 Локальный запуск

```
1 cd weeks/week-17
2 docker compose up --build
```

7.2 Проверка

```
1 # Create user
2 curl -X POST http://localhost:8080/api/users \
3   -H "Content-Type: application/json" \
4   -d '{"name": "alice", "email": "alice@example.com"}'
5
6 # Create comment by exists user (201)
7 curl -X POST http://localhost:8080/api/comments \
8   -H "Content-Type: application/json" \
9   -d '{"author": "alice", "text": "Hello!"}'
10
11 # Create comment by dont exists user (400)
12 curl -X POST http://localhost:8080/api/comments \
13   -H "Content-Type: application/json" \
14   -d '{"author": "nobody", "text": "fail"}'
15
16 # Get all comments
17 curl http://localhost:8080/api/comments
```

8 ЗАКЛЮЧЕНИЕ

В ходе выполнения финального проекта были спроектированы и реализованы два микросервиса (Comments Service и Users Service) на FastAPI, объединённые за Nginx API Gateway. Для межсервисного взаимодействия используется gRPC: при создании комментария Comments Service выполняет gRPC-запрос к Users Service для проверки существования автора. Проект контейнеризирован с помощью Docker (multi-stage сборка) и запускается одной командой `docker compose up`. Система поддерживает полный CRUD для комментариев, базовые операции для пользователей и валидацию автора комментария через gRPC.